0	1	2	3	4	5
2	3	5	4	1	9

⇒ Print

Recursion

2
3
5
4
1
9

```
for( i=0 ;i<n ;i++)
    cout << arr[i];
```

Print karaa o
aa ge bado;

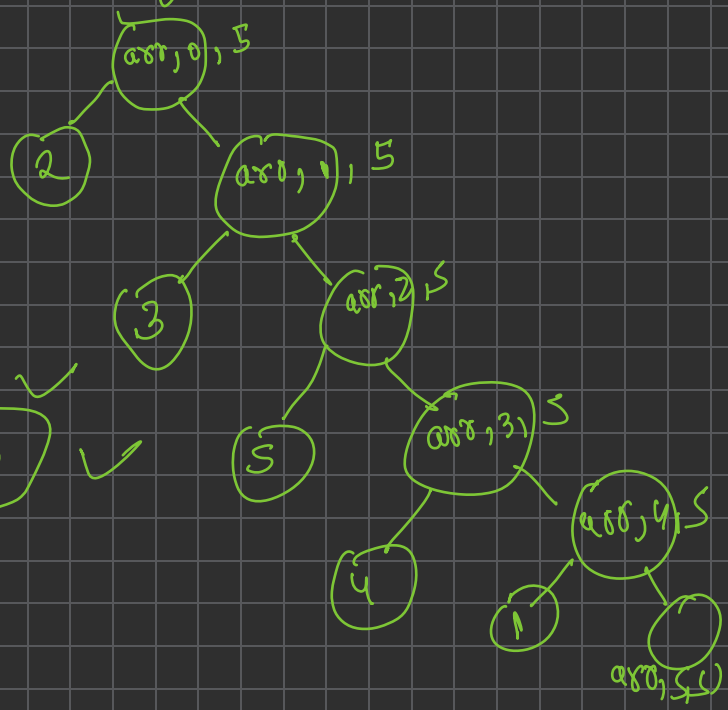
```
void print (int arr[], int index, int n)
{
    if( index==n)
        return;

    cout << arr[index];
    print (arr, index+1, n);

}
```

1
4
5
3
2

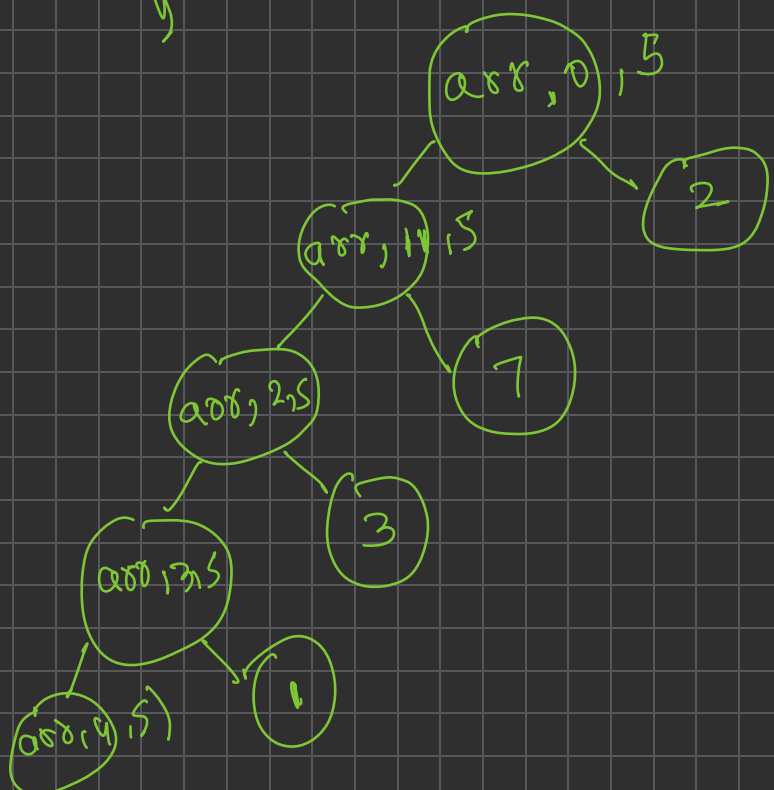
0	1	2	3	4
2	3	5	4	1



1. aage badO
2. print karO

```
void reverse( int arr[], int index,
{       if (index == n) int n )
            return ;
        reverse ( arr, index+1 , n )
        cout << arr [index];
}
```

0	1	2	3	4
2	7	3	1	5



(arr, 5, 5)

5

index == -1

-1

4

0	1	2	3	4
3	1	2	8	5

(1) value ko print karo

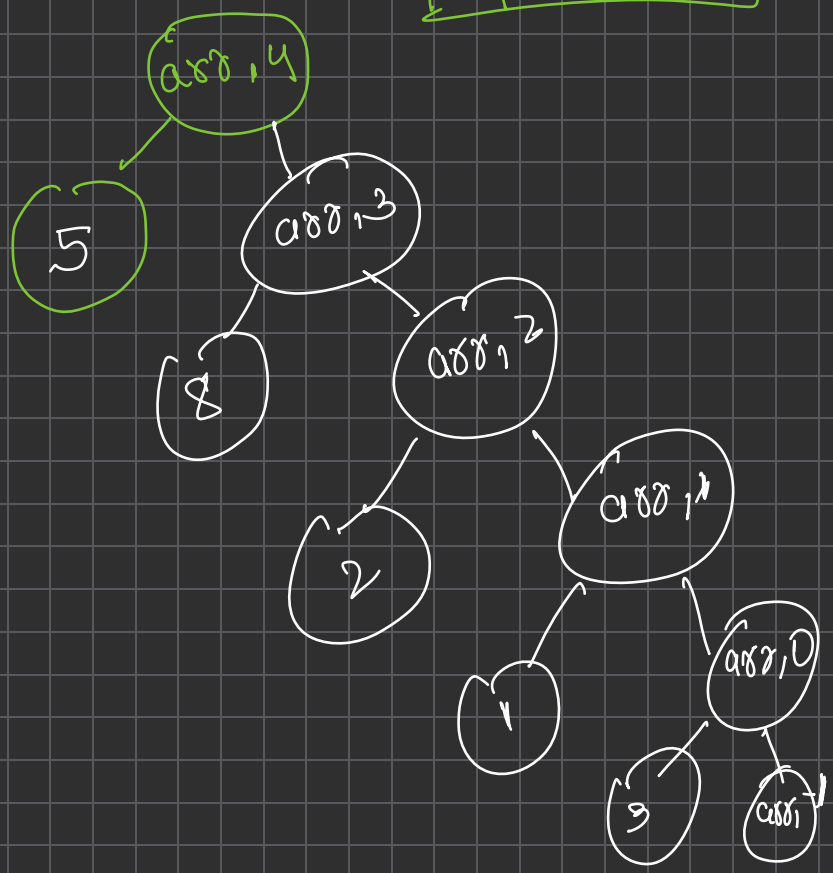
(2) index-1 kar do

```
void reverse(int arr[], int index)
{
    if (index == -1)
        return;

    cout << arr[index]
    reverse(arr, index-1)

}
```

0	1	2	3	4
3	1	2	8	5



0	1	2	3	4
10	3	5	8	2

10
3
5
8
2

```
void print (int arr[], int index)
{
    // if (index == -1)
        return;

    print (arr, index-1)
    cout << arr[index];

}
```

Recursion

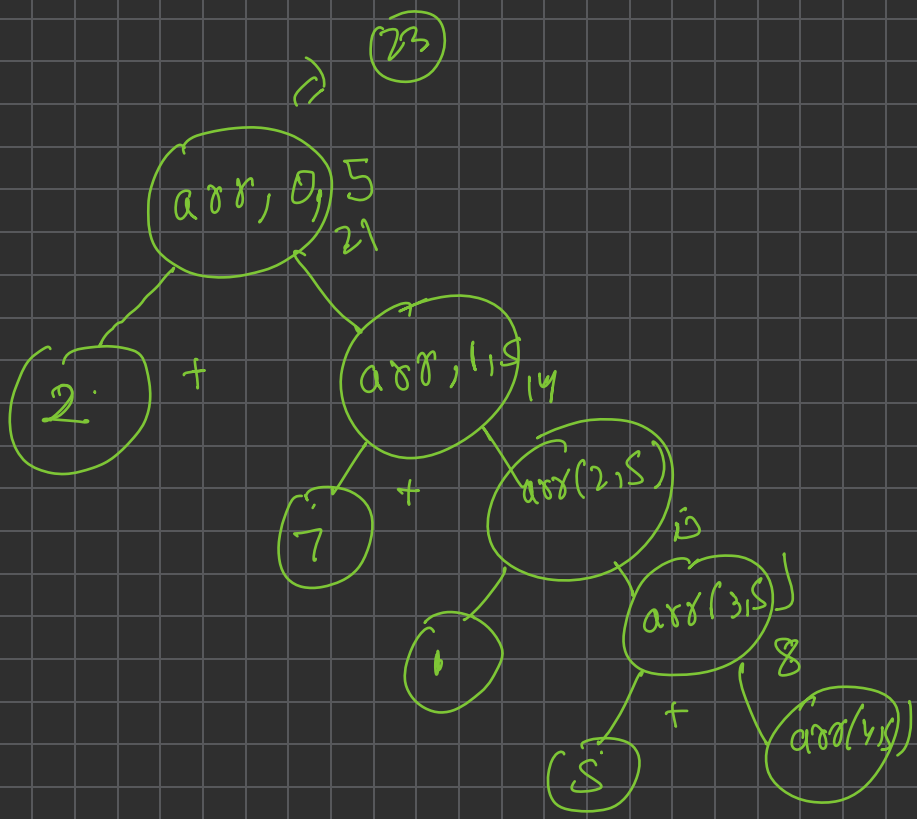
index

0	1	2	3	4
2	7	1	5	8

23

```
int Sum (arr[], index, n)
{
    if (index == n)
        return 0

    int ans = arr[index] + Sum(arr, index+1, n)
    return ans;
}
```



r + arr[s]

String: → pallindrome: ? ✓

naman rohit

racecar racecar

naman

rohir

while(start<=end)

{

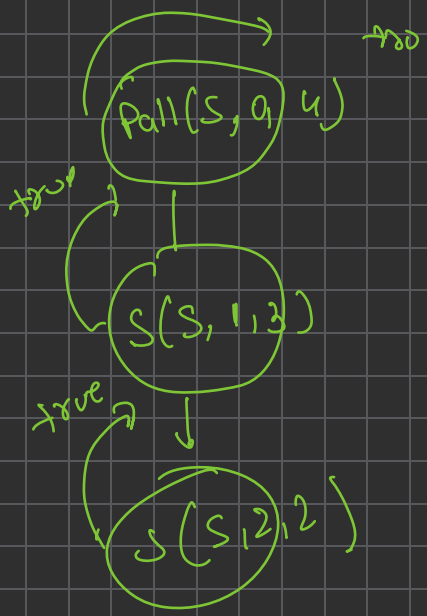
}

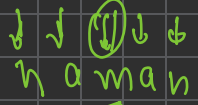
```
bool palli(string &s, int start, int end)
{ ① if(start >= end)
        return true;

  ② if(s[start] != s[end])
        return false;

return palli(s, start+1, end-1);

}
```


Pall(S, 0, 4)
true
true
S(S, 1, 3)
true
S(S, 2, 2)


n a m a n

n o o n

start = 0 ✗ 2

end = 3 ✗ 1

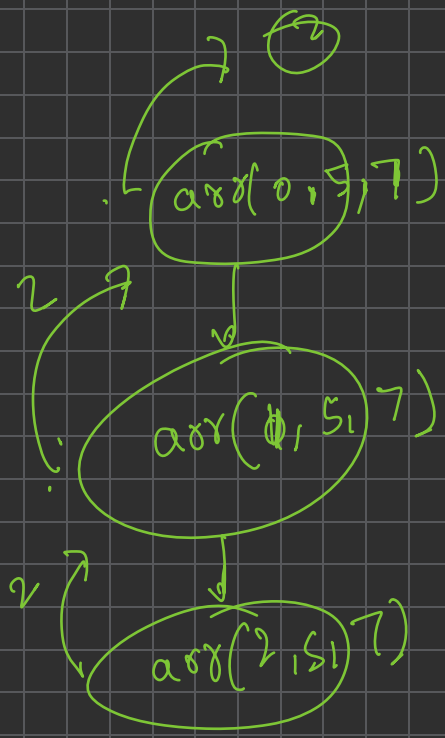
start == end

return

0	1	2	3	4
3	7	1	4	6

Recursion

```
int Find(int arr[], int index, int n,
         int target)
{
    if(index == n)
        return -1;
    if(arr[index] == target)
        return index;
    return Find(arr, index+1, n, target)
}
```



0	1	2	3	4
2	3	7	1	6

target = 7